

VITEGourmand

Documentation de Gestion de Projet

Sommaire

1. Contexte et objectifs	3
1.1 Objectifs du projet	3
1.2 Changement de projet en cours de route	3
2. Méthodologie	3
2.1 Approche et outils	3
2.2 Écart assumé sur le flux Git	4
3. Découpage et planification	4
4. Suivi et pilotage	5
5. Difficultés et solutions	5
6. Bilan et axes d'amélioration	6
Annexes	6

1. Contexte et objectifs

1.1 Objectifs du projet

VITEGourmand est le site d'un traiteur événementiel : vitrine des menus et plats, tunnel de commande en ligne, et back-office d'administration. L'objectif métier est de permettre à un client de composer une commande pour une date de prestation donnée, en tenant compte des allergènes, des délais de préparation et de la zone de livraison, pendant qu'un administrateur gère la carte, les commandes et les avis.

Objectifs fonctionnels :

- Présenter menus, plats, galerie, horaires et zones de livraison.
- Permettre la commande avec date de prestation, délai de commande bloquant, gestion du stock et géolocalisation de la livraison.
- Gérer les allergènes (14 allergènes majeurs, règlement UE n° 1169/2011), les comptes utilisateurs, les avis clients, le suivi de commande et les emails automatiques.

Objectifs pédagogiques de l'ECF : mettre en œuvre une architecture MVC en PHP natif, une base de données hybride SQL/NoSQL, un déploiement en production sur Heroku, un flux Git structuré, et une démarche de sécurisation des accès.

Périmètre : le paiement en ligne n'est volontairement pas inclus dans ce périmètre ECF — une commande est enregistrée avec le statut « En attente » et validée manuellement par la suite.

1.2 Changement de projet en cours de route

La consigne initiale pour cet ECF était de conserver le projet EcoRide (plateforme de covoiturage développée en PHP, MySQL, MongoDB, JavaScript, HTML/CSS et Bootstrap), déjà avancé à environ 70-75 % au moment du changement. À quelques semaines de l'échéance, la consigne a changé : il fallait finalement livrer un nouveau projet, VITEGourmand. Ce changement tardif a provoqué un report de ma part, s'est ajouté une erreur d'inscription de l'organisme de formation sur la session d'examen. Toutes ses erreurs et un retard dans mon organisation du a la panique provoquée par l'imprévu, j'ai du adapter mon fonctionnement.

Face à cette contrainte, la priorité a été donnée au travail concret et au code : terminer, tester et déployer VITEGourmand plutôt que de reconstruire une démarche de gestion de projet complète en partant de zéro. Le tableau Trello initialement conçu pour EcoRide a été réutilisé et adapté (colonnes, cartes, checklists) pour suivre VITEGourmand, plutôt que d'en créer un nouveau un choix assumé pour ne pas ponctionner davantage un temps de maquettage et de développement déjà compté.

Un choix assumé, avec une limite reconnue

Cette réorientation se ressent dans la méthode : le suivi Trello a été recalé après coup sur un développement déjà engagé, plutôt que pensé dès la phase de cadrage comme cela aurait été possible avec un temps de préparation complet sur un projet démarré de zéro. Ce point, et la façon dont il a été géré, sont repris en section 6 (Bilan).

2. Méthodologie

2.1 Approche et outils

Le projet a été mené selon une approche Kanban solo (un candidat, pas d'équipe) combinée à un flux Git structuré de type Git Flow simplifié : une branche principale de production (main, reliée à Heroku), une branche d'intégration (dev), et des branches de fonctionnalité pour les développements ciblés (par exemple feature/allergenes-conditions-menu).

Rituel de travail : découpage en User Stories → développement sur branche de fonctionnalité → test local sous XAMPP → merge dans dev → merge dans main une fois la version jugée stable et livrable.

Outils utilisés :

- GitHub (remote `origin`) pour l'hébergement du dépôt et l'historique de versions.
- Heroku (remote `heroku`) pour le déploiement en production.
- XAMPP en local pour le développement et les tests avant merge.
- Trello pour le suivi visuel des tâches (board adapté depuis EcoRide, voir 1.2 et section 4).
- VS Code comme éditeur, avec Cline utilisé strictement en lecture seule pour auditer et diagnostiquer le code existant — le développement lui-même reste fait à la main.

2.2 Écart assumé sur le flux Git

Sur la dernière ligne droite avant l'échéance, plusieurs commits (correctifs de déploiement, mise à jour du README) ont été poussés directement sur dev et main sans passer par une branche de fonctionnalité dédiée, ce qui rompt ponctuellement le schéma feature → dev → main suivi jusque-là sur le reste du projet. Plutôt que de réécrire l'historique Git pour masquer cet écart, le choix a été de le documenter ici tel quel : il reflète la pression de fin de délai plus qu'un abandon de la méthode.

3. Découpage et planification

Le développement a été découpé en six blocs de travail successifs, du socle technique jusqu'au déploiement et à la sécurisation. Le tableau ci-dessous reprend ce découpage avec, pour chaque bloc, un ou plusieurs commits servant de preuve.

Bloc	Fonctionnalité	Commit(s) de référence	Statut
S1	Socle MVC, base de données et authentification	<code>schema.sql</code> , <code>fixtures.sql</code>	OK
S2	Plats, menus et galerie (colonne <code>ordre</code>)	<code>3abf364</code>	OK
S3	Allergènes et délai de commande bloquant	<code>6590562</code> , <code>574e0fa</code>	OK
S4	Commande : validation JS, gestion du stock, adresse pré-remplie	<code>50d65ce</code> , <code>d7f6429</code>	OK
S5	Géolocalisation de la livraison via API	<code>79764e4</code> , <code>a59361b</code>	OK
S6	Avis MongoDB, emails Mailtrap, sécurisation, déploiement Heroku	<code>da08c0d</code> , <code>6289164</code> , <code>2ac6ff6</code>	OK

Exemples de User Stories :

- « En tant que client, je veux filtrer les plats par allergène, afin d'éviter ceux auxquels je suis allergique. »
- « En tant qu'administrateur, je veux imposer un délai minimum de commande par menu, afin de garantir un temps de préparation suffisant. »

4. Suivi et pilotage

Le pilotage du projet s'est appuyé sur deux outils complémentaires : Trello pour la vue d'ensemble des tâches, et Git (branches, commits, merges) pour la traçabilité technique. Le board Trello reste consultable en direct — plutôt que des captures d'écran figées, le lien d'accès est donné ci-dessous.

Tableau Trello (adapté depuis EcoRide) :

<https://trello.com/invite/b/6819be2af860f1387c78e653/ATTI8c014da2cdb2beff56f69ff21b21ca34884EA745/vite-et-gourmand>

Les colonnes et cartes reprennent la structure initialement pensée pour EcoRide (À faire / En cours / Terminé, checklists par fonctionnalité), réadaptées aux fonctionnalités réelles de VITEGourmand — menus, allergènes, commande, avis, déploiement — plutôt que reconstruites depuis une feuille blanche (voir 1.2).

Indicateurs de suivi :

- Une vingtaine de commits significatifs sur l'ensemble du projet (hors corrections mineures).
- Plusieurs merges `dev` → `main` correspondant chacun à une version jugée stable et déployable.
- Trois correctifs de déploiement identifiés spécifiquement en production (casse de fichier, extension PHP manquante, épingleage de version) — détaillés en section 5.

5. Difficultés et solutions

Certaines difficultés relèvent de l'organisation du projet, d'autres du déploiement technique. Les deux catégories sont regroupées ici pour donner une vue honnête et complète des obstacles rencontrés.

Problème	Cause	Solution	Commit
Changement de projet imposé à environ trois mois de l'échéance	Réattribution tardive du sujet ECF, cumulée à une erreur d'inscription de l'organisme de formation sur la session d'examen	Priorisation du code fonctionnel de VITEGourmand ; réutilisation adaptée du Trello conçu pour EcoRide plutôt qu'un nouveau board construit de zéro	—
Crash en production sur <code>/menus</code>	Extension <code>ext-mbstring</code> manquante dans <code>composer.json</code>	Ajout de la dépendance de plateforme	<code>60851ec</code>
Plantage de <code>ReviewController</code> sur Heroku	Casse de fichier incohérente (Windows insensible à la casse, Linux sensible)	Renommage du fichier pour faire correspondre exactement la casse	<code>6289164</code>
Identifiants de connexion en dur dans le code	Risque de sécurité en cas de fuite du dépôt	Extraction vers <code>constants.local.php</code> (gitignoré) et <code>.env</code>	<code>da08c0d</code>
Adresse non pré-remplie / avertissement PHP	Variable de commande mal initialisée	Correctif ciblé sur le contrôleur de commande	<code>d7f6429</code> , <code>a59361b</code>
Délai de commande non réellement bloquant	La date de prestation restait librement modifiable	Blocage effectif via <code>delai_commande_valeur/delai_commande_unite</code>	<code>574e0fa</code>

6. Bilan et axes d'amélioration

Ce qui fonctionne : le code est livré, testé et déployé sur Heroku ; les identifiants compromis ont été révoqués et régénérés ; les fonctionnalités métier prévues (allergènes, délai de commande, géolocalisation, avis, statistiques) sont opérationnelles.

Limite assumée : la gestion de projet a été rattrapée en cours de route plutôt qu'anticipée dès le cadrage initial — le Trello adapté depuis EcoRide en porte la marque, avec une structuration plus a posteriori qu'un board pensé dès le départ pour VITEGourmand (voir 1.2).

Avec davantage de temps : reconstruire un board Trello dédié dès le cadrage plutôt que d'en adapter un existant, ajouter des tests automatisés (PHPUnit), mettre en place une intégration continue, et intégrer un moyen de paiement en ligne (Stripe).

Enfin, l'échéance du 23 septembre 2026 correspond à un rendu intermédiaire destiné à l'organisme de formation (Studi), et non à la soutenance finale devant le jury de certification, prévue en janvier. Le code et cette documentation sont donc amenés à continuer d'évoluer d'ici là.

Annexes

Liens utiles pour consulter le projet en direct (dépôt, application déployée, suivi de projet) :

Dépôt GitHub : <https://github.com/NKO971/VITEGourmand>

Application déployée (Heroku) : <https://vitegourmand-nicolas-ecf-ddcb2611506c.herokuapp.com/>

Tableau Trello :
<https://trello.com/invite/b/6819be2af860f1387c78e653/ATTI8c014da2cdb2beff56f69ff21b21ca34884EA745/vite-et-gourmand>